MapReduce-Style Computation in Distributed Virtual Memory

Zhiqiang Ma Ke Hong Lin Gu
Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
{zma,khongaa,lingu}@cse.ust.hk

***Abstract*—Many cloud computing technologies, such as MapReduce, use file systems as the system-wide substrate for data handling. A distributed file system provides a global name space and stores data persistently, but it also introduces significant overhead. Several recent systems use DRAM to store data and tremendously improve the performance of cloud computing systems. However, both our own experience and related work indicate that a simple substitution of distributed DRAM for the file system does not provide a solid and viable foundation for data storage and processing in the datacenter environment, and the capacity of such systems is limited by the amount of physical memory in the cluster. To overcome the challenge, we construct a distributed virtual memory to unify the physical memories and disk resources on many compute nodes to form a system-wide data substrate. The new substrate provides a general memory-based abstraction, takes advantage of DRAM in the system to accelerate computation, and, transparent to programmers, scales the system to handle large datasets by swapping data to disks and remote servers. Above the distributed virtual memory, we develop vMR, a new design of the MapReduce framework with enhanced functionality enabled by the memory-based abstraction. The evaluation results show that vMR is faster than Hadoop on all the 4 workloads, and delivers 6-11x speedups on the adjacency list workload. For iterative k-means clustering, vMR can run more than 7 times faster than Hadoop and scale linearly to 160 compute nodes on the TH-1/GZ supercomputer.**

I. Introduction

Modern cloud and big-data processing systems rely on high-performance computation in datacenters. Each datacenter contains one or more clusters composed of a large number of compute servers and a high-bandwidth interconnect, both built upon commodity hardware but often customized. Programming such a large loosely-coupled system is, however, very challenging. Traditional solutions, represented by MapReduce and Hadoop, use a distributed file system to provide a global name space and data substrate [1]–[3]. This enables the system to scale, but makes it slow and difficult to support interactive, real-time or sophisticated computation. More subtly, the file system based data substrate induces designers to assume long latencies when they design other components in the system, making the system even slower.

The reliance on the file system was a reasonable design choice when hard drives were the only economical random-access device for storing hundreds of gigabytes of data [4]. However, the hardware cost structure and technological context today have been dramatically different from those constraining designers a decade ago. Along with several recent research initiatives, we believe that a memory-based abstraction is viable and can significantly enhance the programmability and performance of cloud computing systems. In 2009, Phoenix showed MapReduce computation can use shared memory on a multicore system [5]. In 2010, MRlite implemented in-memory MapReduce computation on multiple compute servers [6], RAMClouds proposed to build DRAM-based storage [7], and Spark is proposed for in-memory computation across a number of compute servers [8]. Today several DRAM-based cluster computing systems, including DVM [9], RAMClouds [10] and Spark/RDD [11], have been implemented and exhibited significant advantages in efficiency and peformance.

However, a simple substitution of DRAM for the file system abstraction cannot provide the functionality and viability required for a solid data substrate on the datacenter platform. Both our own experience and related work indicate that pure DRAM-based systems have the following limitations in comparison to disk-based file systems.

- *Cost and scalability*: Although the price of DRAM has been sharply reduced in the recent decade, so does the price of hard drives. Today the cost of DRAM is still two orders of magnitude higher than that of hard disks for the same capacity. For example, a representative configuration of RAMClouds organizes 64TB storage using 1000 servers [7]. The unit cost, $60/GB based on listing prices, is 500 times higher than that of disks. At the same cost, a disk-based file system can easily provide 4PB storage capacity.
- *Persistency*: A file system stores data persistently but DRAM is volatile. While replicating data in the main memory of multiple servers can emulate persistent data storage, such mechanisms must span wide geographic regions for it to be resilient to correlated faults, such as power outage in a city. The cost of long-distance replication is concerning, and latency multiplies even for in-datacenter replication.
- *Semantic gap*: The role of the file system in the cloud computing technology is, in fact, much wider than providing storage. It exposes a global name space for inter-process communication, enforces atomicity for data writes, and may assist in the concurrency control of the system. For example, multiple tasks on the Google

cluster can exchange data in the GFS name space, and MapReduce uses GFS' atomic rename to implement idempotent reduce semantics. Such file system semantics are sometimes subtle, but they are crucial to the correctness of the computation in a large distributed system. Current DRAM-based storage systems cannot provide such semantics.

We believe that the memory-based abstraction can indeed significantly enhance the capability and performance of cloud computing systems, but the design shall construct a solid data storage and exchange substrate with clearly-defined and useful semantics, and the abstraction does not necessarily fixate on only DRAM-based organizations. In fact, DRAM and magnetic disks have their own unique advantages that complement each other's, and traditional virtual memory systems are an example of integrating the strengths of both to achieve a memory abstraction that is "as fast as memory, as large as the disks". Similarly, a synergy of memories and hard disks, facilitated by today's high-bandwidth network, will provide a fast, scalable and cost-effective way to store and transport data in a cluster.

Hence, we design a distributed memory system called VOLUME (Virtual On-Line Unified Memory Environment) to unify main memories and hard drives on many servers and construct a uniform memory space shared by potentially a large number of tasks. VOLUME handles the data in physical memory when possible, and can also smoothly scale to store large data using disks. Importantly, scaling to handle large data and the optimization of data flows are transparent to the programmers, hiding the fact that data reside on distributed compute nodes. Moreover, VOLUME allows programs to make data persistent, and defines memory operation semantics so that groups of memory operations behave in a way similar to transactions with atomicity, isolation and persistency.

It is also our goal to make VOLUME general-purpose—it should be designed to support a wide spectrum of computation, including not only offline data processing but also low-latency, iterative, interactive and consistency-critical computations. As a general-purpose substrate, VOLUME should readily benefit existing cloud computing technologies without requiring creation of new instruments or an overhaul of the existing architecture—it should be possible to substitute VOLUME for the existing data substrate in a cloud computing technology and immediately enhance its functionality and performance. To verify this and to enhance the widely used MapReduce computation model, we design vMR, an enhanced MapReduce framework that stores and exchanges data on VOLUME.Maintaining the scalability to handle large datasets, vMR demonstrates a flexibility to handle more complex computational logic and an efficiency comparable to recent in-memory cluster computing systems. Designed as a general-purpose data substrate, VOLUME can also be applied in other file system based cluster computing systems besides MapReduce, and accelerate computation without losing the functionality, economy and scalability provided by the file systems. Our performance evaluation shows that VOLUME facilitates low-latency and high-throughput data exchanges, and that the MapReduce implementation based on VOLUME can be 7 time faster than Hadoop on some workloads.

While virtual memory and MapReduce are well understood, designing distributed virtual memory and MapReduce computing over that presents significant technical challenges in several aspects. We highlight the challenges, outline our approaches and state the contributions below.

- VOLUME aims to provide a large uniform memory space in which each processor (node) can address any subset of the memory space. The data a single node accesses may well exceed the size of its main memory and local hard drives combined together, and, consequently, data may reside in local main memory, local hard drives, remote main memory or remote hard drives. VOLUME must transparently fetch data from and swap them to appropriate locations. This is one of the reasons why a simple design that "expands" the memory on individual nodes with large swap areas cannot work for our purpose. VOLUME must hide the location details without introducing excessive overhead, and maintain a uniform global address space with consistent semantics on a distributed system with commodity hardware and unreliable network links. A large computation in a datacenter can potentially scale to thousands of concurrent tasks on hundreds of nodes. To the best of our knowledge, VOLUME is the first "virtual memory" system that coordinates transparent memory accesses to distributed memories and disks at such a scale.
- VOLUME should provide a mechanism for application programs to store data persistently and read data from I/O devices. We introduce persistent memory in the system, and VOLUME automatically writes data to hard drives in a way that allows other tasks or programs to retrieve the data at a later time. This mechanism advances the memory-based cloud technology to systematically specify, store and read persistent data.
- We design VOLUME to provide a transactional behavior and ensure the writes of multiple data by a task are atomic. Our implementation of vMR leverages the atomic guarantee to ensure its correctness. To the best of our knowledge, VOLUME is the first system to provide transactional memory semantics on virtualized memory among many computers.

The rest of the paper is organized as follows. Section II presents the design of VOLUME. Section III shows how to construct vMR on VOLUME. We present evaluation results in Section IV. Section V surveys related work and Section VI summarizes our conclusions.

II. Design

We design VOLUME (Virtual On-Line Unified Memory Environment), a distributed transactional virtual memory system with persistency support, as the substrate of data handling in datacenters. VOLUME provides a unified memory space, and virtualizes distributed physical memory and disks from

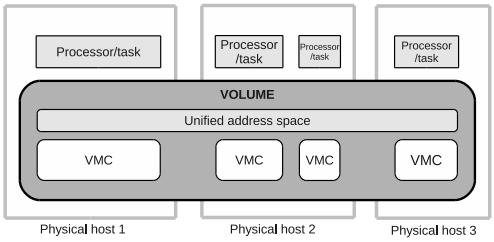


Fig. 1: VOLUME architecture

compute nodes connected through the network. In addition to the physical memory on the local node, the data in VOLUME may reside in the local disks, remote physical memory or remote disks. The data distribution and communication are transparent to programs running in VOLUME. The data access is as efficient as using physical memory when the data is on local physical memory.

In VOLUME, tasks of a program share the same memory space, access their data structures during the execution and make the data persistent if it is needed with normal memory operations, following a snapshot-based transactional semantics. In our implementation, we dispatch tasks to processors for execution using Layer Zero [12] which manages myriads of physical hosts in datacenters to form a big virtual machine and provides a meta-scheduler that schedules tasks to the compute nodes in the system.

A. Distributed Virtual Memory

Figure 1 shows the architecture of VOLUME. VOLUME runs on top of one, multiple or many physical hosts. We abstract groups of memory and disk resources to be *virtual memory containe*rs (VMC). The system has many processors and VMCs. A VMC manages the physical memory and disks in it and provides the virtual memory resource. All the VMCs together form the unified memory system. VMCs handle the resource requests from the upper layer, and swap data among the physical memory, disks, and remote VMCs through the network when it is needed.

As many tasks of a program access the same unified memory space of VOLUME concurrently, it is challenging to efficiently control and support the parallel accesses. VOLUME provides the atomicity and snapshot semantics to programs. Each task can use a specific list of pages grouped to be a snapshot which is a set of memory ranges that are instantiated with the memory state at a particular point of time. Each task's updates to the virtual memory during its execution only affect its own snapshot. A task can only commit its changes in its snapshot to the global memory space when it exits and commits. Hence, tasks are isolated and each has a consistent view of the memory space. After a task exits and commits successfully, its changes are visible to other tasks whose snapshots are created after the commit. For this reason, VOLUME has a home service for coordinating the snapshots and each task has its own page table. The home service maintains the metadata of the memory pages in the whole memory space for creating snapshots and guarantees that atomicity of commit operations. If a task's commit has any write-write conflict with other tasks', the commit fails and the task is aborted.

To assist a task's accesses to the memory space, the VMC for the processor/task makes use of the virtual memory hardware of the host to capture page faults when the task accesses a memory range on a page that is not ever used by it, similar to demand paging. Different from demand paging and traditional distributed shared memory (DSM), VOLUME designs placement and eviction mechanisms for both local and remote memory and both local and remote disks. When handling a page fault, the VMC searches this page in the task's page table which is prepared before the task's execution according to its snapshot and stores the location information of the pages. If the page is in the page table, VMC sets up the page and retrieves the content of the page locally or from a remote VMC, and the task can use this memory page. A task that accesses pages out of its snapshot is aborted.

Locality and scheduling To better control the scheduling of tasks and improve the locality, we add two schedulers for assigning tasks to processors—the deterministic scheduler and the delay scheduler—to Layer Zero's meta-scheduler which uses a FIFO scheduler to assign tasks in the task queue to free processors by default. The deterministic scheduler assigns tasks to processors by hashing the IDs of tasks to processor IDs. With the deterministic scheduler, the scheduling for tasks to processors is deterministic and keeps the same for repeated executions of a program, which makes it easier to analyze the performance. Besides providing repeatability, the deterministic scheduler also gives programmers an instrument to control the scheduling policy of the meta-scheduler. In general, it also improves the locality compared to the FIFO scheduler for jobs that run repeatedly. The more sophisticated delay scheduler uses the delay scheduling techniques [13] and may skip some tasks from the front of the task queue and assign the task that has better locality on a processor/VMC when the processor becomes available. However, if a task at the head of the queue has been skipped for a certain period of time, it is dispatched to an available processor even if the locality was poor. The locality information is collected from the memory range usage history of tasks according to their snapshots when the meta-scheduler dispatches tasks to processors.

B. Persistency

Programs need a mechanism to make some of its data persistent for storing results which may be used by other programs at a later time. Files are the common form of persistent data stored on disks. However, the programmers need first convert the in-memory data structures to a format suitable for a file and copy them from the memory to file systems. The data preparation, system calls and I/O lead to extra performance overhead in both the programs generating the persistent data and the ones using them.

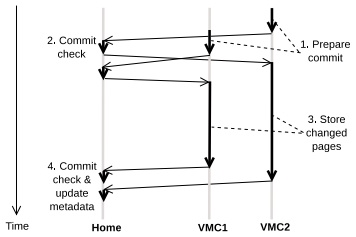


Fig. 2: VOLUME commit protocol. Two commits proceed concurrently.

To provide programmers a flexible and efficient interface to store persistent data, we design the *persistent memory* mechanism in VOLUME. The persistent memory is a memory region which programs can access through regular memory operations. In this way, the distinction between in-memory and persistent data structures is eliminated, and the same set of operations in the programs can be applied to both kinds of data.

To make the state changes of the persistent memory easy to reason about by programmers, the memory ranges in the persisitent memory that a task accesses are also in its snapshot. After the task commits successfully, the changes to the persistent memory by a task in its snapshot are guaranteed to be persistent on disks in the VMCs. Consequently, another task can access and update data in the persistent memory deterministically at a later time.

As the disks are unified in the memory space with the physical memory, we should design the mechanism that can efficiently serves tasks' memory accesses in the persisitent memory with data on local or remote disks. For efficiently finding the data on disks, VOLUME maintains a one-to-one mapping of pages in the memory space to blocks (pages) on disks and the page fault handling mechanism for non-persistent memory ranges can be reused. When a task reads a memory range in the persistent memory for the first time, a page fault is triggered and captured by the VMC. Different from handling the page fault for non-persisitent memory range, the VMC sets up the page and retrieves the content from local or remote disks.

C. Atomic commits

One possible way to implement atomic commits is to let the home service (home) decide whether a commit succeeds or aborts without requiring a distributed atomic commit protocol [14], [15]. The VMC for handling the commit first stores copies of the changed pages locally, and then the home checks whether the commit can be applied and changes the metadata if the commit passes the check.

However, as VOLUME unifies the physical memory and disks, there are challenges for supporting atomic commit efficiently. To handle possible faults of the home, the changes to the persistent memory's metadata should be made persistent atomically if the commit succeeds. Additionally, VOLUME requires saving the changes in the persistent memory to disks which takes more time than storing the data in physical memory. If a commit is aborted by the home, simply saving the changes to disks in the persistent memory before committing is wasted and should be avoided.

Hence, we design the commit protocol of VOLUME, as shown in Figure 2, that detects and aborts conflict commits early and enables the recovery of the metadata of persistent memory if the home service crashes or restarts by storing changes to the persistent memory's metadata to a write-ahead log. Specifically, the protocol for commits consists of the following steps. We use the commit of a task's snapshot by a VMC as the example.

1) The VMC prepares the commit by collecting the metadata of updated pages by the task in its snapshot, and queries the home whether the commit can proceed.
2) The home decides whether the commit should be aborted by checking the in-memory metadata and returns the result to the VMC. This can abort the commits that have conflicts with other applied commits early and avoid useless saving data to disks (*early abort*).
3) If the commit is aborted by the home, the VMC discards the changes. Otherwise, the VMC saves changed pages in the persistent memory to disks, stores copies of changed pages in another memory area, and submits a commit request to the home.
4) The home checks the commit request with the *latest* metadata again. If the commit passes the checking, the home allocates an ID for this commit, writes "*start-commit ID*" and the metadata modifications for the persisitent memory to the write-ahead log, and commands other VMCs that involve in the commit, if there are any, to save pages locally. After all VMCs report that they have completed the commands, the home applies the modifications to the in-memory metadata and logs "*complete-commit ID*" to complete the commit.

With this protocol, the changes made by a task are committed atomically. Any commit that fails to complete the step that the "*complete-commit*" log entry is written is aborted when the home restarts. After a commit succeeds, the changes made are visible to tasks with snapshots created later. All successful commits exhibit a serial order according to the corresponding "*complete-commit*" log entries. In the commit protocol, only a very small part of the total work for a commit is on the home and can finish quickly. Hence, the home can handle many commit requests efficiently.

Using the transactional semantics of VOLUME, a program can also ensure that only one task commits its changes successfully when multiple tasks conduct the same work. One example of these tasks are the backup executions in MapReduce [2]. These tasks share a *status code* s which is the value in a memory range that indicates whether a task has completed successfully. The task, say, T_s, which exits and commits first will commit successfully and atomically, and updates s. The other tasks for this work can be categorized into two classes:

- The task, say, T_1, created its snapshot before T_s commits. The value of s in T_1's snapshot indicates the task is not completed successfully yet. Hence, T_1 proceeds with the computation, updates s and tries to commit. As T_s already successfully committed the ranges that are also updated by T_1, T_1 fails to commit (early aborted).
- The task, say, T_2, created its snapshot after T_s commits. The value of s in T_2's snapshot indicates the task is already done. Hence, T_2 aborts itself.

As both classes of tasks will abort, only one task, T_s, completes the work and commits its results successfully.

D. Handling faults

To help programs recover their execution in the presence of faults, VOLUME handles faults gracefully and provides a slightly weaker yet easy to reason about semantics: after VOLUME handles faults from the home service or VMCs by restarting them, the memory ranges in the persistent memory are recovered with the content updated by the latest successfully committed snapshots before the recovery while the content of other ranges in the memory space is undefined. The atomicity and snapshot semantics of VOLUME make it easy for programmers to design the programs to recover the execution—the content in the persistent memory is always consistent and programs can store necessary state and data in the persistent memory to support the recovery.

Detection of faults for a scalable system should be efficient and lightweight so that it does not leads too much overhead. Faults in VOLUME are detected by making use of existing network traffic among VMCs, home service and the meta-scheduler as confirmations of liveness without additional communication overhead, explicit heartbeats when no enough confirmations are received, and self-reporting from VMCs for certain failures, such as corrupted data. During a recovery of VOLUME, the home service recovers the metadata for the persistent memory from the write-ahead log as discussed in Section II-C and the VMCs recover the saved pages of the persistent memory from the file systems on the hosts where the VMCs reside. The hosts can use either local file systems or networked ones which can ensure very high availability by replicating data to multiple nodes [3], [16].

III. Implement Fast MapReduce on VOLUME

VOLUME provides a flexible and efficient data substrate for the datacenter platform. To investigate how to use distributed virtual memory to enhance the existing design with VOLUME, we develop vMR, which is a new MapReduce framework enhancing the MapReduce programming model, and existing MapReduce programs can be easily ported to vMR. Different from earlier MapReduce designs, the data flows among map and reduce tasks in vMR are handled by the memory abstraction provided by VOLUME, which enables the system to be more efficient and flexible. Such data flows include the reading and parsing of the input, the shuffling of intermediate data and the emission of output data by reduce tasks.

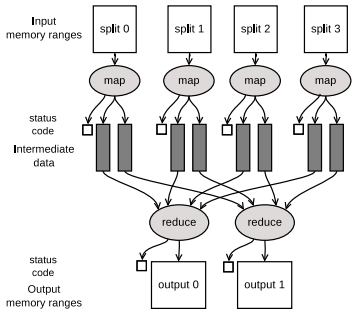


Fig. 3: Data flows among tasks

Execution control vMR handles a job with a set of map tasks and reduce tasks following the MapReduce model. vMR distributes the map and reduce tasks to a plurality of compute nodes to exploit parallelism, and makes sure that tasks execute in a correct order following the dependence relation between map and reduce tasks using *depending task*s `mbarrier` and `rbarrier` that check map and reduce tasks' status codes. A depending task is a special task that is schedulable only after some other tasks commit to its *depended area* which is a set of memory ranges in the memory space. Different from Hadoop and the implementation described in [2], vMR does not require a master program or master node to assign tasks to workers or slave nodes. The scheduling and coordination function in vMR are provided through the tasks and depending tasks and the location of data to process are efficiently and conveniently passed to the tasks as memory addresses in VOLUME.

Data flow The data accessing mechanism plays an important role for the MapReduce systems and is critical for the systems' performance. For example, researchers find that many performance problems of Hadoop can be attributed to its storage system for accessing data [17]–[19]. vMR handles data through VOLUME to support efficient job execution.

To save results which may be required to be stored persistently to be used for other jobs, vMR can be configured to output reduce tasks' results to memory ranges in the persistent memory. Other jobs can access the outputs with normal memory operations. For example, programmers can execute multiple vMR jobs in a chain by using one job's output as another job's input.

Besides the input and output, another dataset during the vMR job is the intermediate key/value pairs generated by the map tasks. MapReduce and Hadoop store the intermediate data on the nodes' local disks by I/O operations which increases the latency of accessing the data. To efficiently maintain and serve the intermediate data, vMR stores the data in the non-persistent region of VOLUME and reduce tasks can access these data with the memory addresses. With memory ranges distributed across many hosts, VOLUME automatically transfers the accessed memory ranges from remote hosts to the host where the task runs. Specifically, a map task retrieves its split of input data from a memory range and stores the *R* pieces of partitioned intermediate data (some pieces

may be empty) into *R* memory ranges, and a reduce task retrieves *M* pieces of intermediate data produced by the *M* map tasks and stores the output into a memory range. The data flows among map and reduce tasks is shown in Figure 3. In our current implementation, the network communication for shuffling and the computation in reduce tasks on different processor/VMCs are overlapped. An alternative method is to implement the shuffle phase as depending tasks checking the map tasks' status codes and collect the intermediate key/value pairs from completed map tasks. By this method, the network communication for shuffling and the map task execution are overlapped.

Hiding details of handling the input, output and intermediate data, MapReduce exploits the restricted programming model to automatically parallelize programs. However, many programs, particularly those with multiple iterations, need to share data among tasks and jobs. On Hadoop, programs use the files in HDFS [20] for sharing data. To support sharing data among tasks and jobs, we design the *shared area* mechanism which is flexible to use and provides strong consistency guarantee based on VOLUME. The shared area $sarea$ is a range of memory for tasks to store shared data. The programs can use it as a program-specific data structure and store and retrieve structured data with low latency. The snapshot semantics of VOLUME makes the state changes of $sarea$ easy to reason about: the changes to $sarea$ by one task is visible to other tasks only after the task atomically commits successfully. Hence, a failed task's changes or a task's intermediate changes to $sarea$ will not be read by other tasks and the $sarea$ is always in a consistent state.

Atomic commits One important semantics of the MapReduce framework is that map and reduce tasks commit their output atomically [2]. This semantics simplifies the failure handling, makes it easy for programmers to reason about programs' behavior, and enables backup executions of tasks to improve performance and reliability. In the design from Dean et al., the atomic commit of the output of a map task relies on the coordination of the master, and the atomicity of reduce task output' commits is guaranteed by the atomic rename operation provided by the underlying file system [2]. vMR ensures the atomic commits of map and reduce tasks using the transactional semantics of VOLUME discussed in Section II-C, requiring neither single master nor assistance from the file system. With the semantics of atomic commits, it is straightforward to implement backup execution in vMR.

Fault tolerance The semantics of VOLUME in the presence of faults discussed in Section II-D enables vMR to handle faults gracefully. The semantics of vMR job executions in the presence of failures is the same as the one in MapReduce [2]. To assist handling faults in vMR, we add the "system idle" signal mechanism which writes the current time to a memory range. The depending tasks in vMR include the memory range for the signal into there depended areas and will be notified when the system is idle. With the signal mechanism, vMR handles the cases of faults as follows.

- A map or reduce task fails. The depending task (`mbarrier` or `rbarrier`) identifies the task as a failed one if it takes much more time than the other map tasks to complete and re-runs the task. The time is read from the memory range for the "system idle" signal. As vMR ensures atomic commits of map tasks, the depending task re-runs the failed task as handling the straggler tasks through backup execution.
- The depending tasks fail. These depending tasks checks the "system idle" signal to ensure that they will be activated again if one instance of the task fails.

Some intermediate data may be lost because of the failures of some nodes. The current implementation handles this fault by terminating the job and reporting the failure to the program which may re-run the whole vMR job. A possible mechanism to handle this more gracefully is that the reduce tasks that read the intermediate data aborts and update their status codes to indicate the failure and the `rbarrier` re-run the map tasks that generate the missing intermediate data and the failed reduce tasks according to the status codes.

IV. Implementation and evaluation

VOLUME is written in C and compiled to x86-64 code. The implementation of vMR comprises 3034 lines of source code. We implement VOLUME and vMR on a cloud computing research testbed and the TH-1/GZ supercomputer which follows the architecture of Tianhe-1A [21]. We measure the performance, scalability and overhead of VOLUME and vMR, and compare the performance on 4 workloads. Hadoop is chosen as the baseline system because it is relatively mature and has been used by many organizations [1]. Following identical algorithmic designs suitable for the MapReduce model, we implement the benchmark programs or use widely used implementations on vMR and Hadoop. The evaluation is conducted on the research testbed using 32 compute servers and on TH-1/GZ using 160 compute nodes.

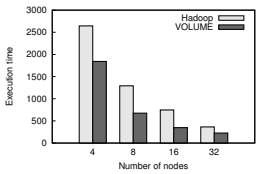
To make the performance comparison fair, we use the same dataset and equivalent job configurations for Hadoop and vMR, in addition to the same implementation and optimization methods of these workloads for both solutions and the same cluster as we run the same workloads. In fact, the configuration slightly favors Hadoop. Hadoop requires static slot allocation on each compute node for map and reduce tasks while vMR can dynamically schedule tasks to available processors to fully make use of the computing capacity. In the experiment, we allocate 1 map task slot and 1 reduce task slot on each compute node for Hadoop, but run only 1 task on each compute node for vMR. With this configuration, the I/O contention is minimized, favoring Hadoop which depends more heavily on I/O. From our evaluation results, we find that, in such configurations, vMR/VOLUME still consistently shows superior performance over Hadoop on all workloads.

A. Workloads

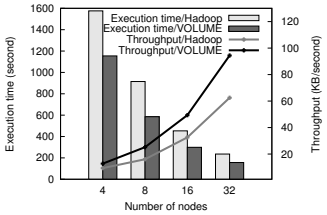
We implement 4 diverse workloads for our evaluation: the *word count*, a widely used benchmark for MapReduce, *k-means* clustering [22], a data mining workload, *adjacency*

Number of nodes	Hadoop (second)	vMR (second)
8	27261.75	3610.83
16	14295.00	1882.50
32	7835.82	1032.26

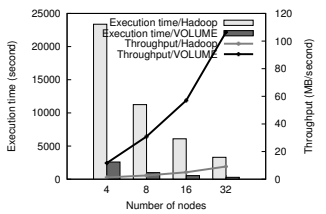
(a) Execution of k-means clustering on 30 million points



(b) Execution time of sorting 50GB data



(c) Execution time and throughput of word count on 15GB input



(d) Execution time and throughput of adjacency list on 30GB input

Fig. 4: Performance comparison

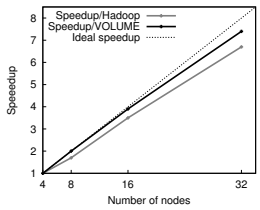
list, a graph processing workload, and *sort*, a widely used benchmark for MapReduce and other systems.

Word count *Word count* counts the number of occurrences of each unique word in a set of documents, and is widely used for evaluation of MapReduce and other parallelization frameworks [2], [5], [23]–[26]. The word count can also represent a large subset of real-world MapReduce jobs that extracts a small amount of interesting data from a large dataset [2]. We implement word count following the implementation described by Dean et al. and the one in Hadoop distribution [1], [2]. The dataset for the word count is the 15GB text content of the static HTML dump of English Wikipedia in June of 2008 [27].

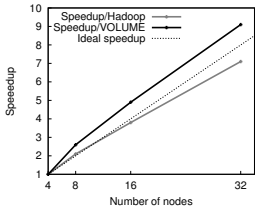
K-means clustering *K-means* clustering is an algorithm commonly used for data mining to iteratively partition a dataset into k clusters [22]. K-means is one example of the set of algorithms that require iterative computation. Our implementations of k-means clustering mainly follow that of Mahout [28], and we adopt relevant optimizations used in related work [29], [30]. The datasets are a large number of 4-dimensional data points which are randomly generated by the standard *gensort* record generator in the TeraSort benchmark [31]. In our evaluation, we cluster the data points into 1000 or 20 groups and make each k-means experiment compute for 20 iterations to limit the execution time.

Adjacency list *Adjacency list* generates the adjacency and reverse adjacency lists of nodes in a graph with the edges as its input. Adjacency list can prepare the graph to be used by PageRank-like computation [32]. The data shuffled during the execution of adjacency list is more than 2 times the size of the input dataset, and the performance of adjacency list can show how vMR performs for shuffle-heavy workloads. The input dataset is a synthetic one from [33] generated following the power law. The datasets we use contains around 100 million nodes (30GB) and 500 million (150GB) nodes with an average out-degree of 7.2. We implement adjacency list on vMR following the implementation on Hadoop from [33].

Sort Sorting is an important part of many computing tasks. Google, Yahoo! and Microsoft also use sorting programs for evaluation in their clusters [2], [34]–[36]. We use TeraSort to evaluate vMR's performance and VOLUME's efficiency as the substrate. Our implementations of the *sort* program on vMR and VOLUME follow the implementation discussed in [2]. The implementation on Hadoop for comparison is the *terasort* program from Hadoop release 1.0.3. We generate the sort program's input consisting of 100-byte records with 10-byte keys by a vMR job, *teragen*, which produces the same dataset for the TeraSort benchmark [31].



(a) Word count



(b) Adjacency list

Fig. 5: Speedups on vMR and Hadoop using increased numbers of compute nodes

Input size	100GB	400GB
Execution and I/O	92.10%	92.68%
Page fault handling	2.07%	0.86%
Home service	1.60%	1.22%
Others	4.23%	5.23%

TABLE I: Execution time breakdown of k-means clustering

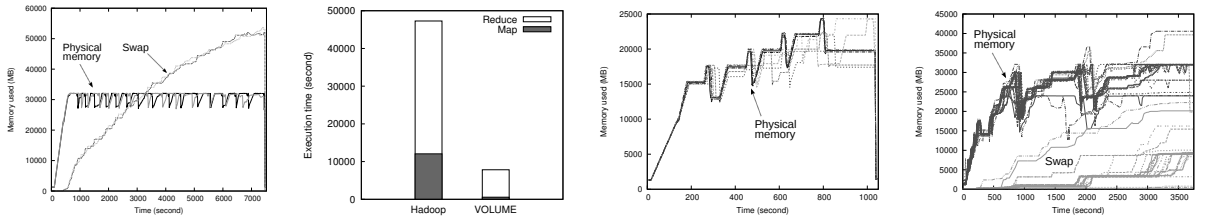
B. Performance comparison

We run sort, k-means, word count and adjacency list on both vMR and Hadoop on the research testbed to compare the performance. Fig. 4 and Fig. 5 show the execution time, throughputs and speedups. vMR delivers higher performance than Hadoop on all workloads. We highlight the performance comparison as follows.

- vMR is more than 7.55 times faster than Hadoop on k-means clustering.
- The sort on vMR is at least 40% more efficient.
- vMR delivers more than 36% higher throughputs on the word count workload and scales out better.
- For the adjacency list workload, vMR delivers throughputs of 6.0 to 11.7 times of Hadoop's and shows super-linear speedups.

As shown in this comparison, VOLUME quickly exhibits superiority in efficiency than the file system based solution for supporting data handling. Specifically, VOLUME accelerates MapReduce computation for the following reasons.

Efficient sharing of data among tasks and jobs. `sarea` is used by k-means on vMR to store the centroids in memory which is far more efficient than reading from the HDFS for k-means on Hadoop.



(a) Adjacency list on 2 nodes (b) Execution time of adjacency list on 2 nodes (c) Adjacency list on 8 nodes (d) Sort 400GB data on 32 nodes

Fig. 6: VOLUME memory usage

Efficiently serving the input and output. We measure the time for reading 1GB data from disks to memory through VOLUME, a native file system and HDFS to compare the overhead from the data handling mechanisms. The times are 12.57, 12.77 and 15.64 seconds respectively, showing that VOLUME is more efficient than the native file system and HDFS for serving data from disks. Zaharia et al. also observe similar results about HDFS' overhead [11].

Low-latency operations on the intermediate data. VOLUME transparently helps vMR save part of the intermediate data generated by map tasks in memory and significantly accelerates data exchange in the shuffle phase. For the shuffle-intensive adjacency list workload, vMR delivers much higher throughputs. The superlinear speedups on vMR is because an increasing quantity of data are stored in memory and exchanged over the network without touching disks as the number of compute nodes grows. This demonstrates VOLUME's efficiency by making use of physical memory well.

Small system overhead. Although VOLUME provides the transactional semantics, the overhead is relatively small. As shown in TABLE I, the time for handling the snapshot on the home service takes only 1.6% of the overall execution time. The efficiency of VOLUME also allows vMR to run jobs with very small overhead. For a no-op job, vMR completes in 0.075 seconds while Hadoop needs at least 23 seconds which is also observed in related work [11].

C. Usage of distributed virtual memory

To evaluate the VOLUME's performance when different amounts of the data are stored on disks, we measure the amount of physical memory and disk space used in the adjacency list workload.

We first stress the VOLUME by running adjacency list on 2 compute nodes that have 64 physical memory in total which is far less than the overall data in VOLUME during the execution of adjacency list. Fig. 6(a) shows the VOLUME usage on 2 nodes. We can see that the physical memory usage increases to around 64GB quickly in around 665 seconds. After that the usage of disk space keeps increasing on both nodes at a lower rate than the previous ones. This shows that the data access speed decreases when the data is swapped to disks. When the data size exceeds the memory capacity, vMR transparently swaps data between DRAM and hard disks, and ensures the success of the adjacency list generation with large datasets.

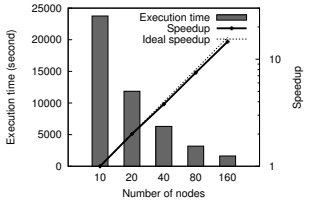
As a large amount of the data is swapped to disks, we compare the execution time of adjacency list on vMR with the execution time on Hadoop. Although disks are highly used by VOLUME to store the data, vMR exhibits more than 6 times higher performance than Hadoop for the adjacency list workload. We also show the time spent in the map and reduce phases in Fig. 6(b). The map tasks on vMR take less than 1/20 of the time that the map tasks use on Hadoop, which shows the efficiency of data accessing in VOLUME. The percentage of time that the reduce tasks use on vMR is much larger than the corresponding percentage on Hadoop. This is because that in the reduce phase, the data in physical memory start to be swapped out to disks and the overall data accessing throughput decrease. However, the reduce tasks on vMR still take much less time than that needed by the reduce tasks on Hadoop, which shows that the cost for memory swapping in VOLUME is smaller than the file system based solution in Hadoop. When there are more than 8 compute nodes, all the data can fit into the physical memory as show in Fig. 6(c). Hence, adjacency list shows superlinear speedups as shown in Fig. 5(b).

D. Scalability

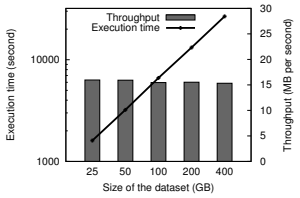
We conduct experiments of k-means and word count on the TH-1/GZ supercomputer to evaluate vMR and VOLUME's scalability up to 160 compute nodes. Fig. 7(a) shows the execution time and speedup of k-means on TH-1/GZ that processes 300 million points. We can see that k-means scales linearly as we scale the compute nodes number from 10 to 160. This presents the good scalability of vMR and VOLUME.

Fig. 7(c) shows the execution time and speedup of word count on TH-1/GZ. The execution time of word count decreases as we scale the number of compute nodes. Word count scales near-linearly on 10 to 40 nodes. However, the speedup is less than the linear one on more than 40 nodes. This is because that the number of reduce tasks in word count is 30 and the additional nodes do not improve the parallelization, and we can see that the time for reduce tasks does not change much on 40 to 160 nodes. However, the execution time of map decreases on more nodes and word count executes faster as the number of nodes increases.

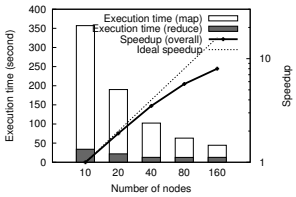
To evaluate how vMR and VOLUME performs as the dataset grows, we run k-means with 30 compute nodes on datasets with varying sizes from 25GB to 400GB. To limit the overall execution time, we set k to 20 for these experiments.



(a) Execution time and speedup of k-means clustering on TH-1/GZ



(b) Execution time and throughput of k-means as the dataset grows



(c) Execution time and speedup of word count on TH-1/GZ

Platform number of nodes	Execution time (second)	Per-node throughput (MB/sec)
Research testbed		
32 nodes	2039.85	2.30
TH-1/GZ		
84 nodes	752.17	2.37

(d) Execution time of adjacency list on 150GB input

Fig. 7: Scalability experiment results

TABLE II: Time for writing persistent data in VOLUME and recovering the home service

Data size	Generating	Commit	Recovery
8TB	6874.8s	129.3s	145.0s

Fig. 7(b) shows the execution time and throughputs. The time increases proportionally to dataset sizes. The throughput remains mostly stable as we scale the dataset to 400GB, indicating that vMR and VOLUME scale smoothly with the data size.

To examine the efficiency of vMR and VOLUME on supporting shuffle-heavy tasks, we run the adjacency list workload with 150GB input. TABLE 7(d) shows the execution time of adjacency list in the research testbed and TH-1/GZ. We can see that the per-node throughput on TH-1/GZ is higher than the one in the research testbed at a larger scale, which shows that vMR and VOLUME can deliver efficient performance and scale well in HPC environments as well as on the cloud computing testbed.

To check how VOLUME performs when the total memory usage is much larger than the overall physical memory and disks are extensively used at a large scale, we scale the input size for sort to 512GB on vMR with 32 compute nodes. TABLE 8(a) shows the execution time and throughputs. As the dataset scales, the throughput decreases gracefully because the usage of disks increases. We measure the VOLUME usage during running the sort on 400GB datasets as shown in Fig. 6(d). From the memory usage, we can see that physical memory of the 32 compute nodes is used up and around 300GB disk space is used for swapping in total when sorting the 400GB data. Hence, the overall data access speed is also affected and the throughput is lower. To look into how the usage of disks affect tasks' execution time, we plot the execution time of tasks during sort 512GB data in Fig. 8(b). Each arrow in the figure represents a task. The map phase consisting of 250 tasks in total complete in 360 seconds, which shows that VOLUME can efficiently serve accessing of input from disks. After that, the reduce tasks start to run. The reduce tasks started after the 2000th seconds take longer time to complete, which demonstrates that VOLUME can smoothly handle large datasets in disks and gracefully decrease the performance.

E. Storing data persistently in VOLUME

To investigate the capability of VOLUME to store a large amount of data persistently and the time for recovering the VOLUME in the presence of faults, we run *teragen* to perform parallel I/O with 8TB data on 30 nodes and force home to restart the VOLUME to emulate crashes. As VOLUME maintains a one-to-one mapping of blocks on disks to memory pages in the persistent memory, recovering the VMCs is very fast. Hence, we only measure the time for recovering the home service.

TABLE II shows the time for generating the datasets and the time used by the home service for committing snapshots and recovering the metadata after being restarted. Although much of the time is for generating the random input, the per-node throughput for generating 8TB data is 38.8MB/s which is about 50% of the raw disk I/O throughput in our cluster. The time used by the home service for committing the 8TB data changes takes 2.8% of the overall execution time, delivering a 61.9GB/s throughput. This shows that the home service is very efficient to support commit requests at a high aggregate rate. For recovery, the home takes 145 seconds for recovering the metadata from the log for persistent memory which stores the 8TB data. Applying certain techniques, such as ckeckpointing, can improve the recovery speed, which we leave as future work.

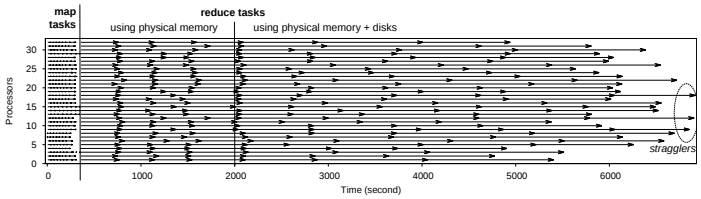
V. Related work

In recent years, MapReduce-style computation is becoming increasingly popular [2]. The MapReduce programming model has greatly simplified the implementation of large-scale data parallel programs and proven to be a useful abstraction for a set of applications [2], [23], [29], [37]–[39]. Since the publication of MapReduce, the research community and industry implemented the MapReduce and its variants, such as Hadoop [1], MRlite [6], Phoenix [25], Mars [40], Twister [41], Map-Reduce-Merge [42], CGL-MapReduce [23], Haloop [43], YARN [44], EMR [45] etc. These variants are used to solve problems in various areas. Different from vMR, these systems follow the design of MapReduce and use networked and local file systems as the substrate. VOLUME, as a general substrate, can provide a flexible support to implement some mechanisms in these variants, such as distributed caching.

The MapReduce systems enable easy development of data processing programs that can scale out to handle very large data usually stored and shared as files in distributed or local

Input size (GB)	Execution time (second)	Through-put (MB/sec)
256	850.7	300.9
320	1969.9	162.5
400	3666.4	109.1
512	6926.6	73.9

(a) Execution time and throughputs



(b) Sort 512GB data

Fig. 8: Execution time and throughputs of sorting on datasets of different sizes on 32 compute nodes and execution progress of sorting 512GB data

file systems. GFS is used as the substrate of the MapReduce in Google's datacenter. Modeled after GFS, HDFS acts as the substrate of Hadoop. However, the performance of disks has not improved so quickly as the disk capacity over the past decades [7] and at the mean time the datacenters are adopting emerging high-performance networks such as 10 Gbps Ethernet [37]. The disk performance is becoming a bottleneck of the MapReduce system, especially for workloads with datasets of moderate sizes. Different from these approaches, VOLUME builds a distributed virtual memory which unifies the physical memory and disk resources as the substrate for computation in datacenters.

While the MapReduce-style programs can scale to handle petabytes of data, there are a large number of jobs that process less than 1TB data which can fit into the physical memory of a cluster of computers. For example, 97% of Hadoop jobs at Facebook handle less than 200GB data with no more than 1500 map tasks (Facebook configures the block size to 128MB) [13]. Dean et al. report that Google often performs MapReduce computation with 200,000 map tasks which process 3.2TB to 12.8TB data (each task is assigned 16MB to 64MB input data), and the average size of data for each job is 114GB to 306GB at Google according to the reported data [2].

In the mean time, the improvement of the DRAM technology enables the construction of data storage completely on DRAM in multiple servers. The compute nodes of one of Yahoo!'s cluster have 16GB physical memory each in 2009, and Ousterhout et al. suggest a server configuration with 64GB memory [7]. Hence, the datasets handled by most of the MapReduce programs can fit into the physical memory of a cluster with tens or hundreds of servers for efficient processing. Researchers suggest using the main memories from many servers in the datacenters to create the storage system [7] and some MapReduce implementations make use of the main memories on working nodes. For example, Twister handles intermediate data in distributed memory, assuming the intermediate data can fit into the distributed memory, and seek to use file systems for processing very large data. Phoenix [25], a MapReduce implementation for multi-core systems, uses memory as its medium for storing intermediate data, while it's input is read from local file systems. Different from vMR and Hadoop, Phoenix can only run on a single node and can not scale to handle larger dataset that can be stored into the memory. Storm [46] is a computation system for continuously processing streams of data which MapReduce processes in batch. However, the capacity of the main memories on the commodity servers are far smaller than the disks' and the storage capacity of these systems is consequently limited. Although a memory-based storage system with large capacity can be built with a large number of servers, the cost is very high and the utilization of the processor resources on these servers may possibly be low most of the time.

Different from MapReduce that exploits a restricted programming model, Dryad takes a more general approach and allows the application running in it to specify an arbitrary "communication DAG" [47]. There are also a number of systems for in-memory computation, such as DSM systems, vNUMA [48], DVM [9], Spark/RDD [11] and Piccolo [49]. Memcached [50], which provides an in-memory key/value store, is widely used to cache back-end database systems for web application. RAMCloud [7] explores the idea that uses physical memory as the first-class medium to store data. Different from VOLUME/vMR, most such solutions do not seek to provide a strong substrate for computation, but target towards loosely-coupled distributed applications such as web serving for their caching or storage systems.

VI. Conclusion

We construct VOLUME, a distributed virtual memory as a system-wide data substrate that provides a general memory-based abstraction with transactional semantics, takes advantage of DRAM in the system to accelerate computation, and, transparent to programmers, scales the system to handle large datasets by swapping data to disks and remote servers. Above the distributed virtual memory, we construct vMR as a new design of the MapReduce framework on VOLUME. VOLUME enables vMR to efficiently handle data with enhanced functionality.

References

[1] "Apache Hadoop," *http://hadoop.apache.org*.

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. of the 6th Symp. on Opearting Systems Design & Implementation*, 2004.

[3] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google file system," in *Proc. of the 9th ACM Symposium on Operating Systems Principles (SOSP'03)*, 2003, pp. 29–43.

[4] L. Barroso, J. Dean, and U. Hoelzle, "Web search for a planet: The Google cluster architecture," *IEEE Micro*, vol. 23, no. 2, pp. 22–28, 2003.

[5] R. Yoo, A. Romano, and C. Kozyrakis, "Phoenix rebirth: Scalable MapReduce on a large-scale shared-memory system," in *IEEE Intl. Sympsium on Workload Characterization*, 2009, pp. 198 –207.

[6] Z. Ma and L. Gu, "The limitation of MapReduce: A probing case and a lightweight solution," in *Proc. of the 1st Intl. Conf. on Cloud Computing, GRIDs, and Virtualization*, 2010.

[7] J. Ousterhout, P. Agrawal, D. Erickson, C. Kozyrakis, J. Leverich, D. Mazières, S. Mitra, A. Narayanan, G. Parulkar, M. Rosenblum, S. M. Rumble, E. Stratmann, and R. Stutsman, "The case for RAMClouds: scalable high-performance storage entirely in DRAM," *SIGOPS Oper. Syst. Rev.*, vol. 43, pp. 92–105, January 2010.

[8] M. Zaharia, N. M. M. Chowdhury, M. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2010-53, May 2010.

[9] Z. Ma, Z. Sheng, L. Gu, L. Wen, and G. Zhang, "DVM: Towards a datacenter-scale virtual machine," in *Proc. of the Eighth Annual Intl. Conf. on Virtual Execution Environments*, 2012.

[10] D. Ongaro, S. M. Rumble, R. Stutsman, J. Ousterhout, and M. Rosenblum, "Fast crash recovery in RAMCloud," in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, ser. SOSP '11, 2011, pp. 29–41.

[11] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. of NSDI '12*.

[12] "Layer Zero," *http://www.lazero.net*.

[13] M. Zaharia, D. Borthakur, J. Sen Sarma, K. Elmeleegy, S. Shenker, and I. Stoica, "Delay scheduling: a simple technique for achieving locality and fairness in cluster scheduling," in *Proc. of the 5th European conference on Computer systems*, 2010, pp. 265–278.

[14] R. Guerraoui, "Non-blocking atomic commit in asynchronous distributed systems with failure detectors," *Distributed Computing*, vol. 15, no. 1, pp. 17–25, 2002.

[15] J. Stamos and F. Cristian, "A low-cost atomic commit protocol," in *Proceedings of the Ninth Symposium on Reliable Distributed Systems*, 1990, pp. 66–75.

[16] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: a scalable, high-performance distributed file system," in *Proc. of the 7th symposium on Operating systems design and implementation*, ser. OSDI '06, 2006, pp. 307–320.

[17] K. Bajda-Pawlikowski, D. Abadi, A. Silberschatz, and E. Paulson, "Efficient processing of data warehousing queries in a split execution environment," in *Proc. of the 2011 Intl. Conf. on Management of data*. ACM, 2011, pp. 1165–1176.

[18] S. Ibrahim, H. Jin, L. Lu, L. Qi, S. Wu, and X. Shi, "Evaluating MapReduce on virtual machines: The Hadoop case," in *Proc. of the 1st Intl. Conf. on Cloud Computing*, ser. CloudCom '09, 2009, pp. 519–528.

[19] J. Shafer, S. Rixner, and A. Cox, "The Hadoop distributed filesystem: Balancing portability and performance," in *Performance Analysis of Systems Software (ISPASS), 2010 IEEE Intl. Symposium on*, march 2010, pp. 122 –133.

[20] "HDFS," *http://hadoop.apache.org/hdfs/*.

[21] X. Yang, X. Liao, K. Lu, Q. Hu, J. Song, and J. Su, "The TianHe-1A supercomputer: its hardware and software," *Journal of Computer Science and Technology*, vol. 26, no. 3, pp. 344–351, 2011.

[22] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. of the fifth Berkeley symposium on mathematical statistics and probability*, vol. 1, no. 281-297, 1967, p. 14.

[23] J. Ekanayake, S. Pallickara, and G. Fox, "MapReduce for data intensive scientific analysis," in *Fourth IEEE Intl. Conf. on eScience*, 2008, pp. 277–284.

[24] S. Huang, J. Huang, J. Dai, T. Xie, and B. Huang, "The HiBench benchmark suite: Characterization of the MapReduce-based data analysis," in *Data Engineering Workshops (ICDEW), 2010 IEEE 26th Intl. Conf. on*, march 2010, pp. 41 –51.

[25] C. Ranger, R. Raghuraman, A. Penmetsa, G. Bradski, and C. Kozyrakis, "Evaluating MapReduce for multi-core and multiprocessor systems," in *Proc. of the 2007 IEEE 13th Intl Symposium on High Performance Computer Architecture*, 2007, pp. 13–24.

[26] M. Zaharia, A. Konwinski, A. D. Joseph, R. Katz, and I. Stoica, "Improving MapReduce performance in heterogeneous environments," in *Proc. of the 8th USENIX Conf. on Operating systems design and implementation*, ser. OSDI'08, 2008, pp. 29–42.

[27] "Wikimedia Downloads," *http://dumps.wikimedia.org/*.

[28] "Apache Mahout," *http://mahout.apache.org/*.

[29] C.-T. Chu, S. K. Kim, Y.-A. Lin, Y. Yu, G. Bradski, A. Y. Ng, and K. Olukotun, "Map-Reduce for machine learning on multicore," in *Proc. of NIPS'07*, 2007, pp. 281–288.

[30] W. Zhao, H. Ma, and Q. He, "Parallel k-means clustering based on MapReduce," in *Proc. of the First Intl. Conf. on Cloud Computiong (CloudCom)*, 2009, pp. 674–679.

[31] "Sort Benchmark," *http://sortbenchmark.org/*.

[32] A. Langville and C. Meyer, "Deeper inside PageRank," *Internet Mathematics*, vol. 1, no. 3, pp. 335–380, 2004.

[33] M. T. Faraz Ahmad, Seyong Lee and T. N. Vijaykumar, "MapReduce benchmarks," *http://web.ics.purdue.edu/~fahmad/benchmarks.htm*.

[34] "Sorting 1PB with MapReduce," *http://googleblog.blogspot.com/2008/11/sorting-1pb-with-mapreduce.html*.

[35] O. O'Malley and A. Murthy, "Winning a 60 second dash with a yellow elephant," *Tech report, Yahoo!*, 2009.

[36] Y. Yu, M. Isard, D. Fetterly, M. Budiu, Ú. Erlingsson, P. K. Gunda, and J. Currey, "DryadLINQ: A system for general-purpose distributed data-parallel computing using a high-level language," in *the 8th Conf. on Symposium on Operating Systems Design & Implementation*, 2008, pp. 1–14.

[37] "Amazon EC2," *http://aws.amazon.com/ec2/*.

[38] Y. Chen, D. Pavlov, and J. F. Canny, "Large-scale behavioral targeting," in *Proc. of the 15th ACM SIGKDD intl conf. on Knowledge discovery and data mining*, 2009, pp. 209–218.

[39] M. C. Schatz, "CloudBurst: highly sensitive read mapping with MapReduce," *Bioinformatics*, vol. 25, pp. 1363–1369, 2009.

[40] B. He, W. Fang, Q. Luo, N. Govindaraju, and T. Wang, "Mars: a MapReduce framework on graphics processors," in *Proc. of the 17th Intl. Conf. on parallel architectures and compilation techniques*, 2008, pp. 260–269.

[41] J. Ekanayake, H. Li, B. Zhang, T. Gunarathne, S. Bae, J. Qiu, and G. Fox, "Twister: a runtime for iterative mapreduce," in *Proc. of the 19th ACM Intl. Symposium on High Performance Distributed Computing*. ACM, 2010, pp. 810–818.

[42] H.-C. Yang, A. Dasdan, R.-L. Hsiao, and D. S. Parker, "Map-Reduce-Merge: simplified relational data processing on large clusters," in *SIGMOD '07: Proc. of the 2007 ACM SIGMOD Intl. Conf. on Management of data*, 2007, pp. 1029–1040.

[43] Y. Bu, B. Howe, M. Balazinska, and M. Ernst, "HaLoop: Efficient iterative data processing on large clusters," *Proc. of the VLDB Endowment*, vol. 3, no. 1-2, pp. 285–296, 2010.

[44] YARN, "Apache Hadoop NextGen MapReduce (YARN)," *http://hadoop.apache.org/docs/r0.23.0/hadoop-yarn/hadoop-yarn-site/YARN.html*.

[45] K.-T. Rehmann and M. Schoettner, "An in-memory framework for extended mapreduce," in *2011 IEEE 17th Intl. Conf. on Parallel and Distributed Systems*, dec. 2011, pp. 17–24.

[46] "Storm," *http://storm-project.net/*.

[47] M. Isard, M. Budiu, Y. Yu, A. Birrell, and D. Fetterly, "Dryad: distributed data-parallel programs from sequential building blocks," in *EuroSys '07: Proc. of the 2nd ACM SIGOPS/EuroSys European Conf. on Computer Systems 2007*, 2007, pp. 59–72.

[48] M. Chapman and G. Heiser, "vNUMA: A virtual shared-memory multiprocessor," in *Proc. of the 2009 Conf. on USENIX Annual technical Conf.*, 2009.

[49] R. Power and J. Li, "Piccolo: building fast, distributed programs with partitioned tables," in *Proc. of the 9th USENIX Conf. on Operating systems design and implementation*, 2010, pp. 1–14.

[50] "Memcached," *http://memcached.org/*.